

Intro to Deep Learning

Lecture 14: CLIP and Multimodal Learning

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What does multimodal learning mean?
2. How does CLIP connect images and text?
3. What is a joint embedding space?
4. How does image-text retrieval work?
5. Why is zero-shot classification possible?
6. Why are prompts important for CLIP?

Lecture rhythm: two encoders -> shared space -> retrieval -> zero-shot -> prompts

Recap From Lecture 09

Visual search used one embedding space:

```
image -> image encoder -> image embedding
```

Then we compared images with cosine similarity.

Today:

```
image -> image encoder -> vector  
text  -> text encoder  -> vector
```

Both vectors live in a shared space.

Part 1

Why multimodal learning?

Why Connect Images And Text?

Many real tasks are naturally multimodal:

- search images with text
- caption images
- classify images using label descriptions
- retrieve products from natural language
- connect documents, screenshots, and diagrams

Text gives flexible descriptions.

Images give visual evidence.

CLIP connects the two.

Part 2

CLIP architecture and shared space

Multimodal Learning

Multimodal learning means learning from more than one data type.

Examples:

```
image + text  
audio + text  
video + text  
sensor data + labels
```

In this lecture:

```
image-text representation learning
```

CLIP Big Idea

CLIP learns to match images with their text descriptions.

It uses:

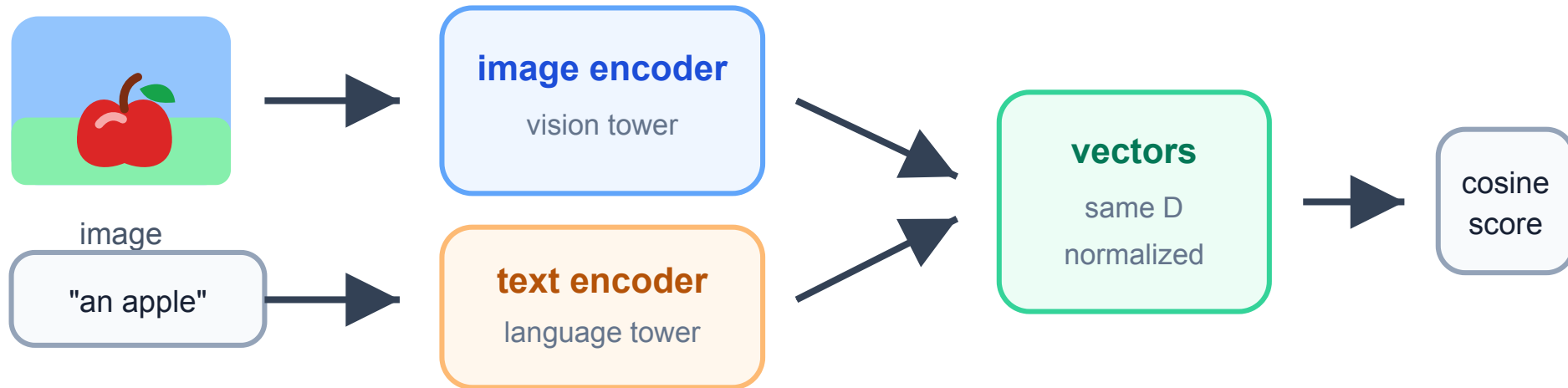
```
image encoder  
text encoder  
shared embedding space  
similarity score
```

If an image and caption match, their embeddings should be close.

If they do not match, their embeddings should be farther apart.

CLIP Two-Tower Architecture

CLIP maps images and text into the same embedding space



Two Encoders

CLIP has two separate encoders.

Image encoder:

turns an image into a vector

Text encoder:

turns a text prompt into a vector

The vectors have the same embedding dimension.

This makes direct comparison possible.

Joint Embedding Space

A joint embedding space means:

images and texts are represented as vectors
in the same coordinate system

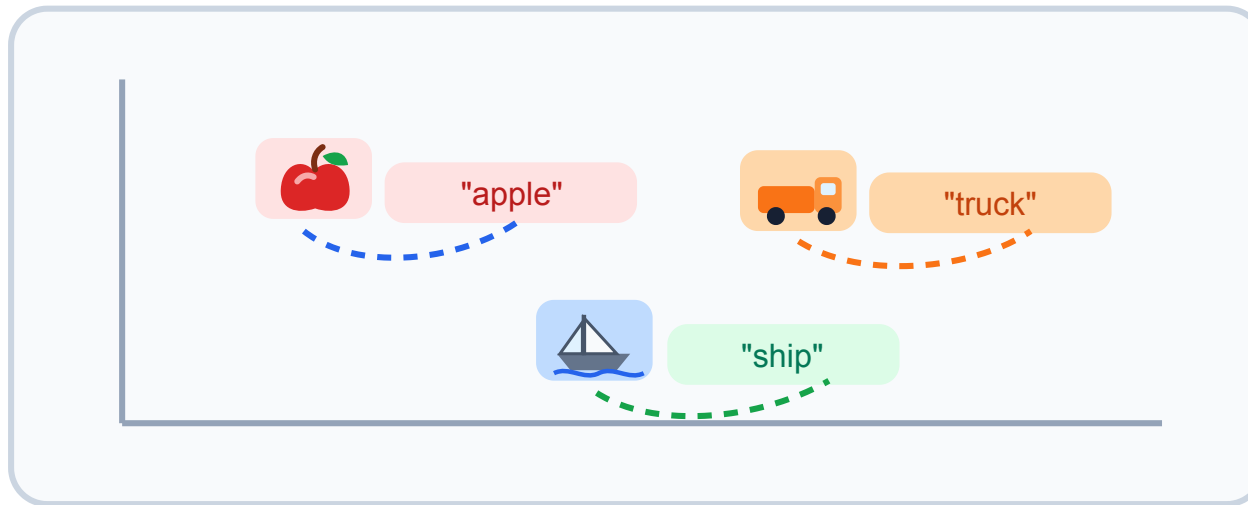
Desired behavior:

image of an apple close to "a photo of an apple"
image of a truck close to "a photo of a truck"

This is the key abstraction.

Joint Embedding Space

Matching images and texts should land near each other



icon = image

label = text prompt

nearby means
high similarity

Similarity

CLIP usually compares normalized embeddings with cosine similarity.

For image embedding i and text embedding t :

$$S = \cos(i, t)$$

With L2-normalized embeddings:

```
cosine similarity = dot product
```

Higher score means a better image-text match.

Similarity Matrix

For many images and many texts:

```
image_emb: [N_images, D]  
text_emb:  [N_texts, D]
```

Similarity matrix:

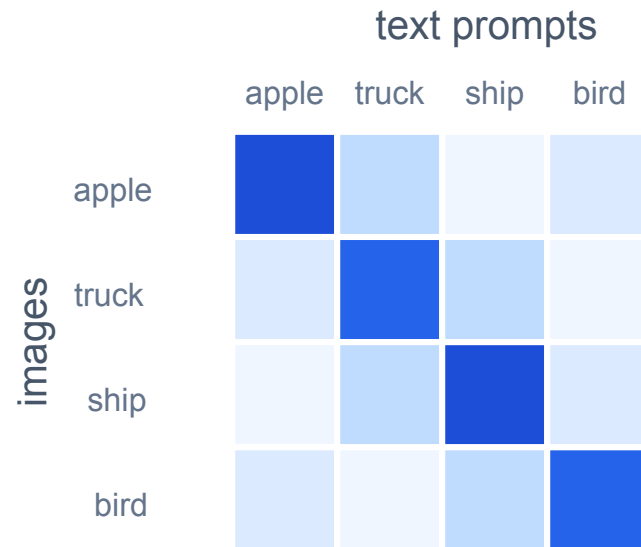
```
S = image_emb @ text_emb.T  
S.shape = [N_images, N_texts]
```

Rows can rank texts for each image.

Columns can rank images for each text.

Similarity Matrix: Picture

Similarity matrix compares every image with every text



bright diagonal
= matching pairs

all off-diagonal cells
are negatives in this batch

How CLIP Is Trained

CLIP is trained on batches of image-text pairs.

For a batch:

```
image 1 matches text 1  
image 2 matches text 2  
...  
image N matches text N
```

Training pushes:

```
matched pairs closer  
mismatched pairs farther apart
```

This is contrastive learning.

Contrastive Learning Recap

For each image:

its paired caption should score highest

For each text:

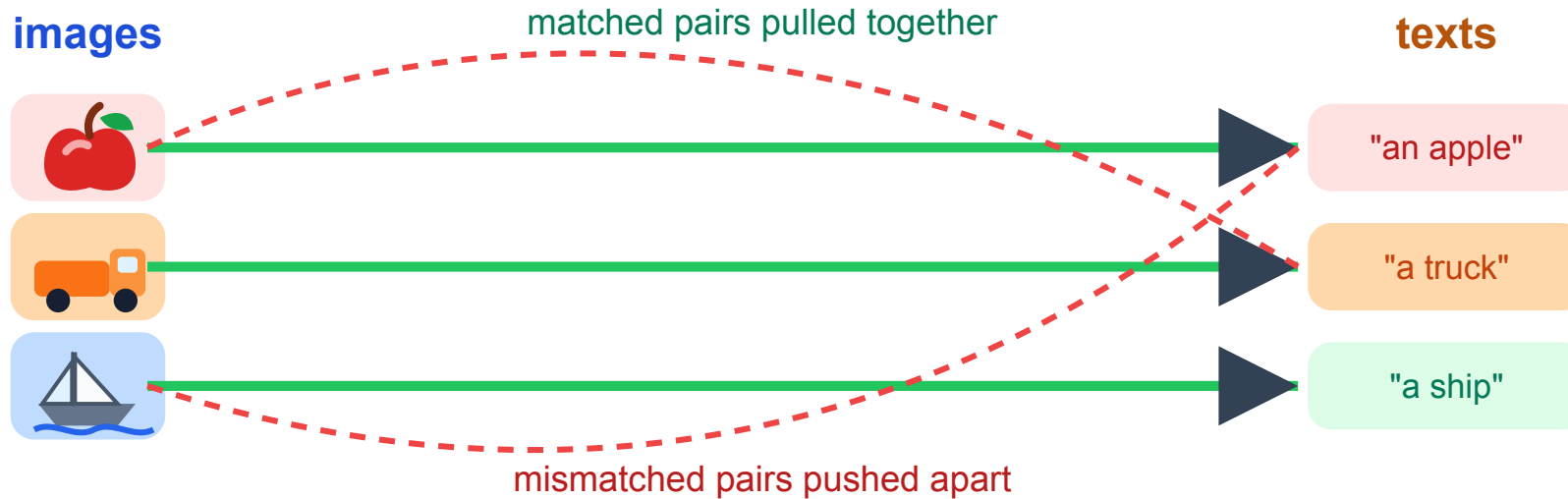
its paired image should score highest

The important idea:

alignment is learned from positive and negative pairs in a batch

Contrastive Batch Recap

CLIP learns by matching paired images and captions in a batch



What The Processor Does

In Hugging Face, CLIP uses a processor.

It handles:

```
image preprocessing  
text tokenization  
padding  
tensor formatting
```

In practice, CLIP is usually loaded as a matched pair:

```
processor = AutoProcessor.from_pretrained(...)  
model = CLIPModel.from_pretrained(...)
```

As usual, processor and model checkpoint belong together.

Embedding Helpers

Useful helper functions usually:

- encode image batches
- encode text lists
- L2-normalize embeddings
- compute cosine similarity

Core pattern:

```
encode -> normalize -> compare
```

This is the same retrieval skeleton as Lecture 09, now cross-modal.

Part 3

Cross-modal retrieval

Text-To-Image Retrieval

Text-to-image retrieval starts with a text query.

Example:

```
"a photo of a red airplane"
```

Pipeline:

```
encode text query  
compare to gallery image embeddings  
return top-k images
```

This lets natural language search an image collection.

Text-To-Image Retrieval

Text-to-image retrieval searches an image gallery with language

top retrieved images

query text

"a photo of a ship"



0.82



0.77



0.51



0.49



0.42

Mini-Check: Retrieval Ranking

You compute:

```
S.shape = [N_images, N_texts]
```

For text-to-image retrieval, you want top images for each text query.

You run:

```
S.topk(k=5, dim=1)
```

What does this rank, and which dimension should text-to-image top-k use?

Answer: Retrieval Ranking

With:

```
S.shape = [N_images, N_texts]
```

`S.topk(k=5, dim=1)` ranks texts for each image.

For text-to-image retrieval, you need top images for each text query.

Two equivalent fixes:

```
S.topk(k=5, dim=0)
```

or:

```
S.T.topk(k=5, dim=1)
```

Image-To-Text Retrieval

Image-to-text retrieval starts with an image.

Pipeline:

```
encode query image  
compare to candidate text embeddings  
return top-k texts
```

Example candidate prompts:

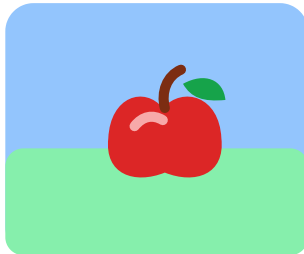
```
"a photo of a cat"  
"a photo of a truck"  
"a photo of an airplane"
```

This is already close to zero-shot classification.

Image-To-Text Retrieval

Image-to-text retrieval ranks candidate prompts for an image

query image



top text prompts

0.84	"a photo of an apple"
0.62	"a photo of a cat"
0.38	"a photo of a truck"

Part 4

Zero-shot classification and prompts

Zero-Shot Classification

Zero-shot classification means:

```
classify examples without supervised training on that dataset
```

With CLIP:

```
turn class names into text prompts  
encode prompts  
encode image  
choose prompt with highest similarity
```

No CIFAR-10 classifier head is trained.

CIFAR-10 Zero-Shot Setup

Class labels become prompts:

```
"a photo of an airplane"  
"a photo of an automobile"  
"a photo of a bird"  
...
```

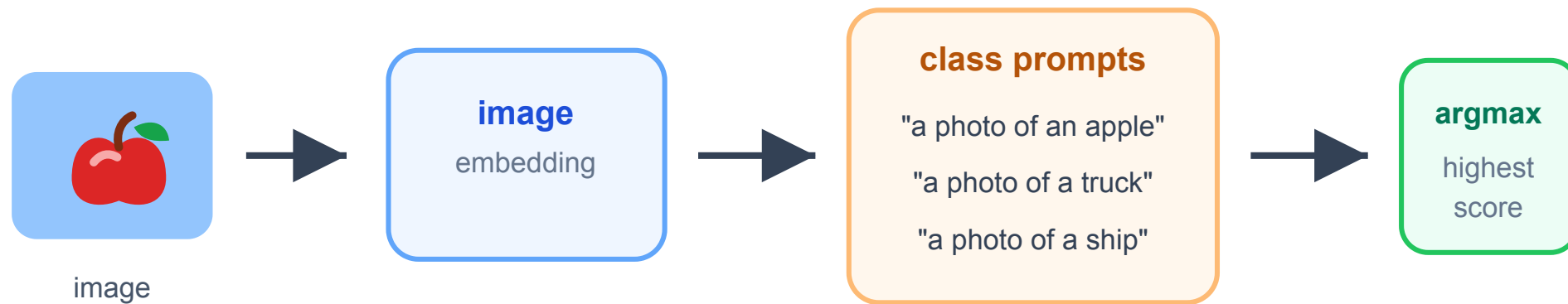
For each image:

```
score image against all class prompts  
predicted class = highest score
```

This is the elegant part of CLIP.

Zero-Shot Classification: Picture

Zero-shot classification is retrieval over label prompts



Why Zero-Shot Works At All

CLIP saw many image-text relationships during pretraining.

So it can connect:

visual patterns
with natural-language concepts

When we write a class prompt, we are using text as a flexible classifier.

The model already has a shared image-text space.

We only provide the candidate labels.

Prompt Sensitivity

CLIP predictions can depend on wording.

Compare:

"cat"

with:

"a photo of a cat"

The second may better match CLIP's pretraining distribution.

Prompt templates can matter a lot.

Mini-Check: Label Mapping

You encode class prompts in this order:

```
cat, apple, truck, airplane
```

But your label IDs expect:

```
airplane, automobile, bird, cat, ...
```

Accuracy looks terrible even though top prompts look reasonable.

What mapping should you inspect?

Answer: Label Mapping

The predicted index comes from the prompt list order.

It does not automatically match the dataset label IDs.

You need to inspect:

```
prompt index -> class name -> dataset label id
```

If prompt order and label order differ, accuracy can look terrible even when the visual-text matching is reasonable.

Prompt Engineering For CLIP

Useful prompt choices:

- include natural context
- match the visual domain
- use class names consistently
- try several templates and compare

Example:

```
"a photo of a {class}"
```

is often stronger than just:

```
"{class}"
```

Prompt Sensitivity: Picture

Prompt wording changes the text embedding

weak prompt

"cat"

less context

stronger prompt

"a photo of a cat"

matches image-caption style

domain prompt

"a CIFAR-10 photo
of a cat"

try and compare

Error Analysis

CLIP errors can be understandable.

Ask:

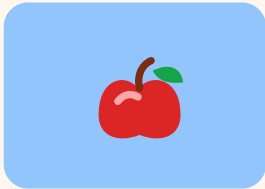
- is the image low resolution?
- is the object small or occluded?
- are classes visually similar?
- is the prompt wording weak?
- is the image unlike CLIP's pretraining data?

For CIFAR-10, resolution and ambiguity matter.

CLIP Error Cases

CLIP errors often reveal ambiguity, resolution, or prompt issues

small object



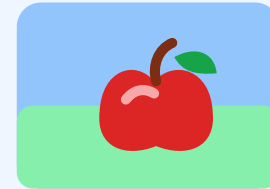
low resolution cue

similar classes



ship vs airplane?

weak prompt



wording matters

CLIP vs Supervised Classifier

A supervised classifier:

```
trained directly on target labels
```

CLIP zero-shot:

```
uses text prompts as labels  
no target-dataset classifier training
```

Supervised models often win on a fixed dataset.

CLIP is powerful because it is flexible across many tasks.

When CLIP Is Useful

CLIP is useful when:

- labels are scarce
- categories change often
- natural-language search is needed
- image-text retrieval matters
- fast prototyping is valuable
- you need a flexible visual representation

It is less ideal when:

you need maximum accuracy on one fixed supervised task

Limitations

CLIP can struggle with:

- counting
- fine-grained visual differences
- spatial relations
- OCR-heavy images
- domain-specific imagery
- prompt sensitivity
- dataset and cultural biases

It is a strong representation model, not a perfect visual reasoner.

Common Mistakes

Mistake 1:

using image and text embeddings without normalization

Mistake 2:

mixing up image-to-text and text-to-image matrix dimensions

Mistake 3:

using processor from a different checkpoint

Mistake 4:

assuming zero-shot means always accurate

Debugging Checklist

Before retrieval or zero-shot classification, check:

- model is in `eval()` mode
- processor matches model checkpoint
- image embeddings shape is `[N_images, D]`
- text embeddings shape is `[N_texts, D]`
- embeddings are L2-normalized
- similarity matrix has expected shape
- class prompt order matches label order

Practical Workflow

A typical CLIP workflow:

1. load pretrained CLIP and processor
2. compute normalized image and text embeddings
3. perform text-to-image retrieval
4. perform image-to-text retrieval
5. run zero-shot classification
6. compare prompt templates

Main idea:

images and text can be compared in one embedding space

Takeaways

CLIP connects vision and language through shared embeddings.

Key ideas:

- image and text encoders map into one space
- cosine similarity compares cross-modal representations
- contrastive learning aligns matched image-text pairs
- retrieval works in both directions
- zero-shot classification is retrieval over label prompts
- prompts and errors deserve careful inspection

Next lecture: